UADE

Trabajo Práctico

Carrera: Ingeniería en Informática

Materia: Programación III

Turno/Comisión: Viernes Tarde

Profesora: Leon Maria Angela

Integrantes:

- **Belanger, Tomas 1192961**
- **Piñango Antonio 1161399**
- **Soto Juan Pablo 1194889**

Fecha de Entrega: 23/10/2026

Informe - Primera Entrega: Planificación de Rutas y Distribución de Ayuda

Decidimos elegir la propuesta 2. El informe cubre las tres funcionalidades obligatorias: organización y consulta de pedidos (F1), ruta más rápida (F2) y selección de cargas prioritarias (F3), y la identificación de las tres funcionalidades elegidas para la Etapa 2.

F1. Organización y consulta de pedidos

Análisis del problema

Cada Pedido tiene su id, centro de destino, y fecha limite (entero AAAAMMDD, para que el orden numérico coincida con el orden cronológico sin parsear fechas), prioridad y peso total.

Se necesitan dos órdenes totales distintas sobre el mismo conjunto (fecha límite ascendente, prioridad descendente) con reglas de desempate explícitas, y una búsqueda por id. El volumen de pedidos crece con el tiempo, así que el costo de ordenar y buscar debe escalar mejor que lineal/cuadrático.

Sin usar Arrays.sort, Collections.sort ni Arrays.binarySearch: hay que implementar el ordenamiento y la búsqueda a mano.

Estrategia elegida y justificación

Usamos **Divide y Conquista** para ambos órdenes, implementando **merge sort**. Se descartó un ordenamiento cuadrático (inserción, burbuja) porque no escala con "cantidades crecientes de pedidos" como pide la consigna; **merge sort** garantiza O(n log n) en el **peor caso** (a diferencia de quicksort, que degrada a $O(n^2)$) y es **estable**, lo cual simplifica el desempate: alcanza con una comparación compuesta sin necesidad de lógica adicional para preservar el orden relativo en caso de empate total.

Para la búsqueda por id usamos **búsqueda binaria**, O(log n), que requiere el arreglo ordenado por id. Como ese orden no es ninguno de los dos pedidos por el enunciado, se calcula aparte y se cachea: la primera búsqueda paga el ordenamiento (O(n log n)); las siguientes, mientras el conjunto de pedidos no cambie, solo pagan O(log n).

Organización de los datos

- Para los dos listados: no se requiere estructura auxiliar persistente, el merge sort trabaja sobre copias del arreglo de entrada en cada llamada.
- Para la búsqueda: un arreglo ordenado por id, calculado una sola vez y reutilizado (atributo de instancia ordenPorIdCache, invalidado implícitamente porque OrganizadorPedidos encapsula una copia inmutable del conjunto de pedidos recibido en el constructor).

Complejidad temporal

Operación	Complejidad
ordenarPorFechaLimite / ordenarPorPrioridad	O(n log n) tiempo, O(n) espacio auxiliar
buscarPorId (primera vez)	O(n log n)
buscarPorId (subsiguientes, sin cambios)	O(log n)

Resultados de las pruebas

Ver Pruebas.java (ordenarPorFechaLimite_desempataPorPrioridadYLuegoId, ordenarPorPrioridad_desempataPorFechaYLuegoId, buscarPorId_existenteYInexistente, organizador_listaVaciaYUnElemento, organizador_noModificaElArregloOriginal). 13/13 pruebas pasan (mvn test). Se verificó a mano que los desempates (prioridad luego id para el orden por fecha; fecha luego id para el orden por prioridad) coinciden con el orden esperado, que listas vacías y de un elemento no rompen el algoritmo, y que el arreglo original pasado al constructor nunca se modifica.

F2. Ruta más rápida

Análisis del problema

La red vial es un grafo **dirigido** con tiempos de viaje **positivos** y fijos durante cada consulta. Hay que encontrar el camino de menor tiempo total entre un origen y un destino, poder deshabilitar una conexión y recalcular, informar cuando no hay ruta, y exponer las decisiones principales tomadas durante el cálculo.

Estrategia elegida y justificación

Usamos **Greedy**, mediante el **algoritmo de Dijkstra**. En cada paso se "cierra" (finaliza) el vértice no visitado con menor distancia tentativa conocida, sin reconsiderar esa elección nunca más. Esa decisión greedy es **correcta,** precisamente porque todos los pesos de las aristas son positivos: no puede existir un camino más corto hacia ese vértice pasando por un vértice aún no cerrado, dado que cualquier camino alternativo solo podría sumar más tiempo.

Descartamos **Bellman-Ford** (admite pesos negativos, pero es O(V·E), innecesario acá) y una búsqueda exhaustiva tipo backtracking (explota combinatoriamente y no hace falta, porque no se piden todos los caminos sino el mínimo).

Implementamos la extracción del vértice de menor distancia con un **min-heap binario propio** (arreglo dinámico + siftUp/siftDown), en vez de java.util.PriorityQueue, con **lazy deletion**: al mejorar la distancia de un vértice no se busca ni actualiza su entrada existente en el heap (costoso), sino que se inserta una entrada nueva; las entradas obsoletas se descartan al extraerlas, comparando su distancia contra la mejor distancia conocida en ese momento (dist[v]) y contra si el vértice ya fue visitado.

Deshabilitar una conexión solo marca la arista (habilitada = false) sin tocar la estructura del grafo; Dijkstra ignora las aristas deshabilitadas al relajar, así que un recálculo es simplemente volver a correr el algoritmo sobre el mismo grafo. Funciona como un “soft delete”.

Organización de los datos

- RedVial: lista de adyacencia (Map<String, List<Arista>>) más una lista de vértices para asignar índices 0..n-1 usados en los arreglos dist, previo y visitado de Dijkstra.
- Arista: origen, destino, tiempo y la bandera habilitada, mutable para poder deshabilitar/habilitar sin reconstruir el grafo.
- Traza: lista de strings que registra cada extracción del heap y cada relajación (mejoró o no), expuesta en ResultadoRuta.traza().

Complejidad temporal

Con el min-heap propio: **O((V + E) log V)**, cada vértice se inserta en el heap al menos una vez por cada relajación exitosa que lo mejora (como máximo una vez por arista entrante), y cada inserción/extracción del heap es O(log V) (V = cantidad de entradas acumuladas en el heap, que en el peor caso es O(E)).

Resultados de las pruebas

Ver Pruebas.java (rutaMasRapida_origenIgualDestino, rutaMasRapida_caminoMinimoVerificadoAMano, rutaMasRapida_sinRutaTrasDeshabilitarUnicaConexion, rutaMasRapida_aristasSonDirigidas). Se verificó a mano, sobre el grafo de ejemplo de Main.java (Depósito, CentroA, CentroB, CentroC, CentroD), que el camino mínimo Depósito→CentroD es Depósito→A→B→C→D con tiempo 20 (10+3+5+2), mejor que las alternativas Depósito→A→C→D (10+20+2=32) o Depósito→B→C→D (15+5+2=22); y que tras deshabilitar B→C, el nuevo mínimo pasa a ser Depósito→A→C→D (32). También se verificó el caso origen = destino (tiempo 0), el caso sin ruta disponible, y que las aristas son dirigidas (B→A no existe si solo se cargó A→B).

F3. Selección de cargas prioritarias

Análisis del problema

Es el problema de la **mochila 0/1**: cada Lote tiene un peso entero positivo y una prioridad entera no negativa, no se puede fraccionar y se usa a lo sumo una vez. Hay que maximizar la prioridad total sin superar la capacidad C del vehículo.

Estrategia elegida y justificación

Aplicamos **Programación Dinámica**. El problema tiene **subestructura óptima** (la solución óptima para n lotes y capacidad C se construye a partir de soluciones óptimas de subproblemas con n-1 lotes) y **subproblemas superpuestos** (la misma combinación (i, c) aparece en múltiples ramas de decisión), lo que hace rentable memorizar en una tabla en vez de recalcular.

Descartamos **Greedy** por razón prioridad/peso (tomar primero los lotes con mejor prioridad por kilo): es óptimo para la mochila **fraccionaria**, pero no para la variante **0/1**, porque un lote con peor razón puede ser imprescindible para aprovechar mejor la capacidad restante exacta. Un enfoque de **backtracking puro** (probar todos los subconjuntos) sería exponencial, $O(2^n)$, y la tabla de PD lo evita acotando el espacio de estados a n × C.

Organización de los datos

Tabla V[i][c] de tamaño (n+1) × (C+1), donde V[i][c] es la máxima prioridad alcanzable usando los primeros i lotes con capacidad c:

V[0][c] = 0

V[i][c] = V[i-1][c] //si peso(lote_i) > c

V[i][c] = max(V[i-1][c], V[i-1][c-peso(lote_i)] + prioridad(lote_i)) //en otro caso

La selección final se reconstruye recorriendo la tabla desde V[n][C] hacia atrás: si V[i][c] != V[i-1][c], el lote i fue incluido y se retrocede a V[i-1][c - peso(lote_i)].

Complejidad temporal

O(n · C) en tiempo y en espacio (pseudo-polinomial: depende del valor de C, no solo de la cantidad de lotes n).

Resultados de las pruebas

Ver Pruebas.java (mochila_casoDeEjemploResueltoAMano, mochila_capacidadCeroONingunLoteCabe, mochila_todosLosLotesCaben, mochila_pesoSeleccionadoNuncaSuperaCapacidad). Se verificó a mano que con capacidad 50 y lotes (10,60), (20,100), (30,120), la combinación óptima es (20,100)+(30,120) = peso 50, prioridad 220 (incluir el lote de peso 10 en lugar de alguno de los otros dos siempre da un total menor). También se verificaron los bordes: capacidad 0, todos los lotes caben, y que el peso total seleccionado nunca supera la capacidad dada.

Funcionalidades seleccionadas para la Etapa 2

Para cubrir la cuarta estrategia pedida por la cátedra, **Backtracking**, y completar el conjunto de seis funcionalidades exigido por la consigna, el equipo eligió:

La funcionalidad A1, **Consultas de rutas entre todos los centros**, (bloque A) Su estrategia prevista es Programación Dinámica (Floyd-Warshall). La elegimos porque reutiliza directamente el grafo (RedVial) ya construido para F2, y responde múltiples consultas de tiempo mínimo sin recalcular todo cada vez.

La funcionalidad B3, **Rutas alternativas con restricciones**, (bloque B) Su estrategia prevista es Backtracking (enumeración con poda). Es un backtracking clásico: un recorrido DFS que descarta ramas por tiempo máximo excedido o ubicaciones prohibidas, aplicado sobre redes chicas donde el resultado se puede verificar a mano.

La funcionalidad B4, **Planificación de entregas con menor costo**, (bloque B) Su estrategia prevista es Backtracking (branch and bound con poda). La propia consigna pide explícitamente "evitar continuar con asignaciones parciales que no puedan completar una solución válida o mejorar la mejor solución encontrada", es decir, poda por cota, que es la variante optimizadora de backtracking. Además, conecta con el dominio de Pedidos y vehículos ya trabajado en F1 y F3.

Alternativas consideradas y descartadas para el bloque A:

A2 - Asignación de vehículos a recorridos*:* también es un buen candidato a Backtracking con poda (similar a B4), pero se descartó para no concentrar las tres electivas en la misma estrategia y priorizar la cobertura de Programación Dinámica fuera de F3.

A3 - Entrega de vales con disponibilidad limitada*:* se puede aplicar Programación Dinámica (similar a una mochila acotada), pero se descartó por ser más redundante conceptualmente con F3 que A1, que además reutiliza infraestructura ya construida.

Alternativas consideradas y descartadas para el bloque B:

B1 - Recorrido con múltiples entregas (TSP)*:* buen ejercicio de Backtracking/poda, pero de mayor complejidad de implementación y verificación que B3/B4.

B2 - Distribución de cargas entre vehículos*:* también viable por Backtracking, pero B4 cubre un problema de asignación con optimización de costo más completo y mejor vinculado a las funcionalidades ya implementadas.

Link del repositorio de GitHub: https://github.com/tomaxbel/TPO_ProgramacionIII_UADE